R0 — C, do PC ao micro

Binário, máscaras, o primeiro código gravado e o código que veio antes do seu

Microcontroladores — DENE/UFMT

OBJETIVOS

- Escrever a mesma lógica em Python e em C, e reconhecer o que muda quando o destino é um microcontrolador.
- Converter entre binário, hexadecimal e decimal para valores de oito bits, e ligar, apagar, inverter e testar um bit sem alterar os vizinhos.
- Compilar e gravar pelo bootloader, e identificar por sintoma as falhas mais comuns do caminho.
- Provar na bancada que `#pragma config` não tem efeito neste kit, e ler os bits de configuração que o bootloader deixou.

SEM NOTA

Este roteiro não vale nota, e é a única sessão do semestre em que isso é verdade junto com o R1.

Ele existe para que o erro aconteça aqui. Escreva máscaras erradas de propósito, troque a ordem das linhas, grave o arquivo errado. A partir do R2 cada roteiro vale nota, e o formato é o deste: pré-lab antes, no máximo três tarefas na bancada, e uma linha do seu código defendida no fim.

PERIGO

Uma regra, e ela vale o semestre inteiro: **a garra de terra da ponta de prova vai num ponto de GND, e em nenhum outro lugar.** Prendê-la num ponto de 12 V — LAMP, HEATER, COOLER — curto-circuita 12 V contra o terra da rede.

Hoje você não usa osciloscópio nem nenhum desses pontos. O aviso vem agora porque o hábito se forma antes de haver risco, e porque, cumprida essa regra, medir é seguro — o que importa muito neste curso.

NESTE KIT

Configuração de bancada para esta sessão:

Chaves SWITCHS (PORTB)	Todas em OFF
CH2-1 (LCD)	OFF
CH3-3 e CH3-5	OFF
Demais bancos	OFF

Os únicos periféricos usados hoje são os oito LEDs.

1 A mesma lógica, três vezes

Vocês já escreveram Python. O mesmo raciocínio, escrito de três formas, mostra o que a mudança de destino custa.

```python
# Python, no computador
ligado = False
while True:
    ligado = not ligado
    print("LED", "aceso" if ligado else "apagado")
    time.sleep(0.5)
```

```c
/* C, no computador */
#include <stdio.h>
```

```c
int main(void) {
    int ligado = 0;
    while (1) {
        ligado = !ligado;
        printf("LED %s\n", ligado ? "aceso" : "apagado");
        /* dorme meio segundo */
    }
}
```

```c
/* C, no microcontrolador */
#include <xc.h>
#include <stdint.h>
void main(void) {
    LATD  = 0xFF;              /* estado antes de virar saida */
    TRISD = 0x00;              /* agora sim: saida            */
    while (1) {
        LATD ^= 0b00000001;    /* inverte o bit 0             */
        for (uint16_t i = 0; i < 30000u; i++) { }   /* atraso  */
    }
}
```

CONCEITO

Três diferenças, e nenhuma é de sintaxe.

Não há para onde retornar. `main` não termina. Se o laço acabar, o processador continua buscando instruções no que vier depois — que não é código seu.

Não há `print`. A saída é um pino. E escrever num pino é escrever numa posição de memória: `LATD` é o nome de um endereço, e o encontro 1 mostra qual.

Não há `sleep`. Ninguém está esperando por você. O atraso é um laço que gasta instruções, e o tempo que ele consome é consequência — não escolha.

2 Pré-lab: antes da sessão

O pré-lab é feito em casa, no caderno, e chega pronto à bancada. Hoje ele é só papel; a partir do R2 ele inclui uma função testada no simulador.

2.1 Bases: por que hexadecimal

Um registrador de oito bits tem 256 estados. Escrevê-los em decimal esconde exatamente a informação que interessa: **quais bits estão em 1**.

Binário	Hexa	Decimal	Leitura
`0b00000001`	`0x01`	1	só o bit 0
`0b00001000`	`0x08`	8	só o bit 3
`0b10000000`	`0x80`	128	só o bit 7
`0b00001111`	`0x0F`	15	os quatro bits baixos
`0b11110000`	`0xF0`	240	os quatro bits altos
`0b11111111`	`0xFF`	255	todos

Cada dígito hexadecimal vale exatamente quatro bits, o que torna a conversão mecânica: parta o byte em dois grupos de quatro e traduza cada um.

TAREFA

Pré-lab 1. Complete sem calculadora.

Binário	Hexa	Decimal
`0b00100000`		

	`0x24`	
		192
`0b01010101`		
	`0xC3`	

ATENÇÃO

O tamanho de `int` não é definido pela linguagem C: varia entre plataformas. Código escrito com `int` funciona numa arquitetura e falha silenciosamente em outra. Neste curso a largura é sempre explícita: `uint8_t` (0 a 255) para registradores e máscaras, `uint16_t` (0 a 65535) para leituras do conversor e temporizadores, `int16_t` para temperatura em décimos de grau.

2.2 As quatro operações que resolvem o semestre

```c
#include <stdint.h>

#define BIT(n)  (1u << (n))

uint8_t r = 0b00000000;

r |=  BIT(2);              /* liga o bit 2, preserva o resto  */
r &= ~BIT(2);              /* apaga o bit 2, preserva o resto */
r ^=  BIT(2);              /* inverte o bit 2                 */

if (r & BIT(2)) {          /* testa: verdadeiro se o bit 2 e 1 */
}
```

CONCEITO

Reconheça o padrão, em vez de decorar as linhas: **ligar é OU com a máscara; apagar é E com a máscara invertida; inverter é OU-exclusivo; testar é E seguido de comparação.**

O `~` do apagar é o erro mais comum de todos. Sem ele, `r &= BIT(2)` zera todos os outros bits em vez de preservar — e o sintoma, "mexi num e sumiram os outros", é inconfundível depois que se vê uma vez.

Não confunda `&` com `&&`, nem `|` com `||`. Os dobrados são **lógicos**: devolvem 0 ou 1. `PORTB && 0x04` quase nunca é o que você quis escrever, e o compilador não reclama.

TAREFA

Pré-lab 2. Partindo de `uint8_t r = 0b10110100;`, escreva o valor de `r` depois de cada linha, **em sequência** — cada uma opera sobre o resultado da anterior.

1. `r |= BIT(0);` _____
2. `r &= ~BIT(4);` _____
3. `r ^= 0b00001111;` _____
4. `r = BIT(1);` _____

A quarta é a pegadinha. Explique numa frase por que ela é diferente das três anteriores.

TAREFA

Pré-lab 3 — a função. Escreva no caderno a função que inverte só o LED `n`, sem tocar nos outros sete:

```c
void alterna(uint8_t n)
{
    /* uma linha */
}
```

e o laço do pisca que a usa para piscar o LED 0. Na bancada você vai digitar exatamente o que trouxer escrito.

3 Na bancada

Existem dois caminhos para pôr código dentro do chip, e este curso usa os dois em momentos diferentes. CONCEITO

Bootloader. Um programa que já está gravado na Flash e que, ao ser acionado, escuta a USB e regrava o resto da memória. É o caminho de vocês. Não precisa de hardware nenhum além do cabo.

Gravador. Um aparelho que fala com o chip por cinco fios e escreve na Flash diretamente, **inclusive os bits de configuração**. É o caminho da demonstração do encontro 0, e o único possível num chip virgem.

Para entrar em modo bootloader: segure **SW9** e energize a placa. NESTE KIT

É SW9, e não SW1. O manual da Exsto diz SW1 e está errado — a serigrafia da placa tem precedência sobre o manual.

Tarefa 1 — o primeiro código gravado. Crie o projeto no ambiente, selecione o PIC18F4550, digite o pisca com a sua função `alterna` e grave pelo bootloader. TAREFA

1. Anote o tempo total do ciclo: da edição do arquivo até o LED responder. Esse número vai importar quando o programa tiver quatrocentas linhas.
2. Troque o laço do pisca por uma linha fixa: `LATD = 0b11110000;`. Antes de gravar, desenhe os oito LEDs no caderno e marque quais você **espera** que acendam. Grave e compare.
3. Troque a ordem: `TRISD` antes de `LATD`. O painel muda? Se não mudou, isso **prova** que a ordem não importa? Responda em uma frase.

O item 2 provavelmente não fez o que você previu. EXPERIMENTO

Não tente explicar agora, e não peça a explicação. Registre com precisão qual byte você escreveu e quais LEDs acenderam, e formule uma hipótese sobre o **circuito** — não sobre o seu código — que explique a diferença. O R1 começa exatamente por aí.

3.1 O código que veio antes do seu

O bootloader é um programa que roda **antes** do seu, toda vez que a placa liga. Ele decide se fica escutando a USB ou se entrega o controle ao seu código. E ele já tomou, de fábrica, decisões que o seu programa não consegue desfazer.

Os **bits de configuração** moram numa área separada da memória, a partir do endereço 0x300000. Eles escolhem o oscilador e os divisores do relógio, ligam ou não o cão de guarda, decidem se o pino 1 é reset. Em C, eles se escrevem com `#pragma config`, e o compilador os coloca no `.hex`. CONCEITO

Quem grava essa área é o gravador. **O bootloader não grava.** Ele gravou a dele uma vez, de fábrica, e não aceita mudanças pela USB.

Tarefa 2 — o `#pragma` ignorado. Acrescente ao topo do pisca a linha TAREFA

```
#pragma config PLLDIV = 5, CPUDIV = OSC1_PLL2, USBDIV = 2
```

que pede ao chip um relógio de processador **três vezes mais rápido** que o atual.

1. **Antes de gravar**, escreva: se a linha tiver efeito, o pisca fica mais rápido ou mais lento, e quanto?
2. Abra o `.hex` gerado num editor de texto e procure uma linha que comece com `:020000040030`. Ela anuncia que os bytes seguintes vão para o endereço 0x300000. A configuração **está no arquivo**?
3. Grave pelo bootloader e observe o pisca. O que aconteceu? Explique com as duas respostas anteriores.

TAREFA

Tarefa 3 — lendo o que o bootloader deixou. O próprio programa consegue ler os bits de configuração. A função abaixo vem pronta; você não precisa entendê-la hoje — ela usa a leitura de tabela, assunto do encontro 1.

```
static uint8_t config_le(uint32_t endereco)
{
    TBLPTRU = (uint8_t)(endereco >> 16);
    TBLPTRH = (uint8_t)(endereco >> 8);
    TBLPTRL = (uint8_t)endereco;
    asm("TBLRD*");
    return TABLAT;
}
```

Grave um programa que mostre nos LEDs o byte em 0x300000 (`CONFIG1L`), com `LATD = ~config_le(0x300000);`. O `~` está lá porque os LEDs deste kit acendem com 0: assim, **LED aceso é bit em 1**.

Anote o byte em binário e decodifique os três campos com a tabela:

Bits	Campo	O que escolhe
5	`USBDIV`	De onde vem o relógio da USB
4–3	`CPUDIV`	Por quanto o relógio do processador é dividido
2–0	`PLLDIV`	Por quanto o cristal é dividido antes do PLL

Consulte a folha de dados (seção de bits de configuração, registrador `CONFIG1L`) para o significado de cada valor. Os números que você anotar hoje são o ponto de partida do encontro 2, onde eles viram os 16 MHz do processador.

3.2 Falhas por sintoma

Sintoma	O que costuma ser
`undeclared identifier 'uint8_t'`	Faltou `#include <stdint.h>`
`'LATD' undeclared`	Faltou `#include <xc.h>`, ou o dispositivo do projeto está errado
`expected ';'`	O ponto-e-vírgula que falta está na linha **anterior**
O gravador não encontra a placa	Não entrou em modo bootloader: SW9 pressionado **antes** de energizar
Compila, grava, e nada muda	Gravou um arquivo antigo — confira a data do `.hex`

NOTA

Leia sempre a **primeira** mensagem de erro, não a última. O compilador perde o fio depois do primeiro tropeço e passa a reclamar de linhas que estão perfeitas. Corrigir a primeira costuma apagar dez.

4 Registro

TAREFA

Entrega. Leve o caderno com: o pré-lab, o desenho dos LEDs esperados contra os observados e a hipótese sobre o circuito, a resposta da Tarefa 2 com a linha do `.hex`, e o byte de `CONFIG1L` decodificado.

Devolva a bancada ao estado da tabela de configuração.

TAREFA

Para o próximo roteiro. Instale em casa o PICSimLab e o MPLAB X com o XC8. A partir do R2, o pré-lab inclui uma função que você testa no simulador antes de chegar à bancada — e a sessão não tem tempo para instalação.

NOTA

No R1: o kit por dentro, com o multímetro. As chaves DIP, o que cada uma liga e desliga, e a resposta da sua hipótese — por que o painel não fez o que o byte dizia.